字符串常量为什么不能改：字符指针的隐藏约束

嵌入式 Linux 手记　2026-10-15　公众号文章 · 原链接 mp.weixin.qq.com

先看四行关于字符串常量的代码。① 和 ② 长得像双胞胎，一个会崩，一个安然无恙：

```
char *s = "hello";
char t[] = "hello";

s[0] = 'H';    /* ① */
t[0] = 'H';    /* ② */
```

① 写的是指针指向的那个字符，运行时行为未定义，典型表现是段错误；② 往自己的数组里写一个字符，天经地义。篇 1 的避坑清单里埋过一句话："这两行代码天差地别，值得单独一篇。"这一篇就是欠条兑现，也是前七篇的收束。

一、一崩一活：类型与存储都不同

```
#include <stdio.h>

int main(void)
{
    char *s = "hello";
    char t[] = "hello";

    printf("sizeof(s) = %zu\n", sizeof(s));   /* 8：一个指针 */
    printf("sizeof(t) = %zu\n", sizeof(t));   /* 6：整个数组 */
    t[0] = 'H';
    puts(t);                                  /* Hello */
    return 0;
}
```

`char t[] = "hello"` 的初始化动作是拷贝（C11 §6.7.9：字符数组允许用字符串字面量初始化）：字面量的 6 个字节（含结尾 `'\0'`）被复制进栈上的新数组，`t` 拥有一份自己的副本，改它是改自己的东西。`char *s = "hello"` 里没有拷贝发生：`s` 只是一个 8 字节指针，存的是字面量本体（躺在另一块内存里）的首地址。

字面量本体值得单独看清：`"hello"` 是一个没有名字的数组，`sizeof("hello")` 的结果是 6 而不是 8，数组身份当场暴露。编译器用它初始化的那块存储具有静态存储期（C11 §6.4.5）：生命周期从程序启动到退出，不随任何函数出入。这一点解释了一个你可能早已见怪不怪的现象：

函数里 `return "hi"` 是合法的，返回的不是栈上地址，函数返回后依然有效。你每次写 `printf("hello")`，传给它的也是这个匿名数组的首元素地址。

类型也早就分家了：`s` 是 `char *`，`t` 是 `char[6]`。篇 1 的 sizeof 铁证在这里再次生效，sizeof 一个是 8、一个是 6，编译期就能把双胞胎拆穿。顺带回答一个高频追问：`s` 和 `t` 的值相等吗？不等。`s` 存的地址在只读节，`t` 的地址在栈上，相隔十万八千里——值相同只发生在"数组名 vs 首元素"那种场景（篇 1 的 `a` 与 `&a`），这里是两个不同的对象。

还要补一句关于 ① 的实话："行为未定义"这五个字里藏着 UB 的阴险之处。同一个程序，Linux 桌面上大概率段错误；某些嵌入式工具链会把字面量放进 RAM，写入可能"看起来改成功了"。测试通过不代表正确，换个平台就炸，这是 UB 类 bug 的标准剧本。



图：`char t[]` 把字面量拷贝进栈上数组；`char *s` 只存一个地址，指向只读节

二、只读的两层防线，都不在类型系统里

先看一个反直觉的事实：C 标准里字符串字面量的类型是 `char[N]`，没有 const 限定（C++ 才是 `const char[N]`）。那"不能改"从哪来？C11 §6.4.5 写得很干脆：程序若试图修改这类数组，行为未定义。这是整个标准里少有的"没有 const 却不可写"。

标准为什么宁可宣布 UB，也不把话说死？因为字面量放哪儿、怎么放，全部留给了实现：可以进只读节，可以烧进 flash，可以在无 MMU 的小平台上直接落进 RAM，也可以被去重合并。标

准不肯替实现选边，就只能把修改行为逐出"可讨论"的范围。从此编译器拿到一纸授权，可以假定你的程序从不改字面量，并据此做一切优化。

这纸授权落地时变成两道防线。

第一道在存储位置。有 MMU 的平台（跑 Linux 的开发板、服务器），字面量通常被链接器安排进 `.rodata` 节；程序加载后，这一段内存页被映射为只读，写入当场触发页异常，也就是篇 3 讲过的段错误。验证不费劲：

```
objdump -s -j .rodata a.out | tail -3
# 68 65 6c 6c 6f 00   "hello."    ← 字面量本体躺在只读节里
```

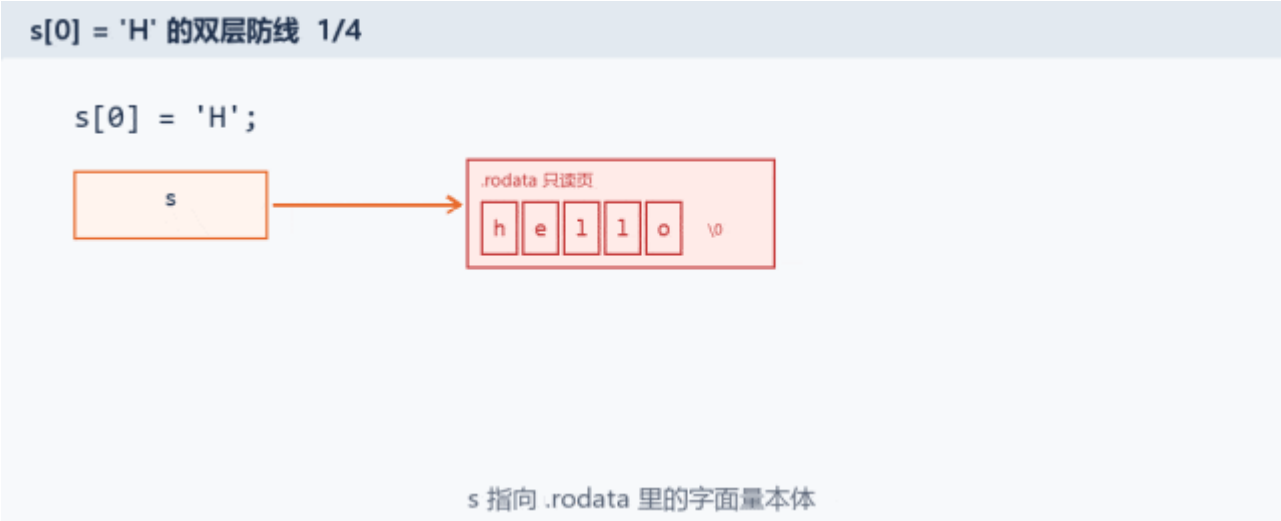
第二道是合并权。C11 §6.4.5 还规定：内容相同的字面量是否共享同一份存储，由实现决定。注意标准用词的微妙：这里写的是 unspecified（未指明），不是 undefined（未定义）。前者是标准摆出两种以上可能、把选择权交给实现（实现通常会把它写进文档）；后者是完全无约束。合并与否由实现拍板，修改字面量则完全无界，两级处置，分寸不同。这次合并看得见摸得着：开 `-O2` 跑下面两行，很多平台会打印同一个地址：

```
printf("%p\n", (void *)"hello");   /* 两次可能打印同一个地址 */
printf("%p\n", (void *)"hello");   /* 字面量合并发生了 */
```

合并的风险值得具象成一个小剧本。假设某无 MMU 平台把字面量放进了可写的 RAM，而编译器把程序里所有内容相同的 `"hello"` 合并成一份：

```
char *banner = "hello";
char *prompt = "hello";
banner[0] = 'H';     /* 假设这台机器写入不崩 */
puts(prompt);        /* 打印出 "Hello" */
```

`prompt` 从头到尾没人动过，打印出来却变了：`banner` 和 `prompt` 指向的是同一份存储。改一个字面量，全程序所有 `"hello"` 一起遭殃，这种看不见的连锁反应，正是标准把修改判成 UB 的现实理由之一。



图：只读的双层防线：只读页保护与字面量合并，以及 const 把崩溃提前到编译期

为什么标准不干脆给字面量加上 const？历史包袱：const 是 ANSI C 才引入的关键字，当时存量代码里 `char *s = "hi"` 满天飞，改字面量类型等于破坏世界。C++ 没有这份包袱，直接断了腕；连"字面量转 `char *` "这个转换本身，C++ 也早早标记为弃用，g++ 不加任何选项都会对它告警。同一个字面量在两种语言里类型不同，根源在此。

三、工程现场：把"只读"提前到编译期

真实 bug 往往绕一道弯：函数形参写成 `char *s`，调用方传进来的是字面量。编译器一声不吭（`char *` 接收字面量在 C 里合法），函数体内一写，运行时才崩。一个典型剧本：日志函数 `log_time(char *tag)` 里想把标签末尾补个冒号，测试环境传的是字符数组，一切正常；上线后有人传了字面量，当场崩溃。这类"测试时好好的，上线才炸"的问题，相当一部分是某处签名少了 const。

修法是把接口签名改成 `const char *s`：既是"我不改"的文档承诺，也让函数体内的写操作直接变成编译错误。C 标准库的 `strcpy` 原型就是这个思路：`char *strcpy(char * restrict dst, const char * restrict src)`，src 一侧带 const，声明"我只读你"；dst 一侧不带，授权"我要写你"。签名即契约，标准库自己就在用。

```
const char *s = "hello";    /* 现代正解：误写直接编译报错 */
/* gcc -Wwrite-strings：给存量代码做体检，char* 接字面量全部现形 */
```

`-Wwrite-strings` 也有一笔代价要算清：它会把字面量按 `const char[]` 对待，存量 C 代码里 `char *s = "..."` 的写法比比皆是，一开就是满屏告警。所以它只适合当一次性的体检工

具，没法默认常开。

需要可写副本时，两种姿势：小而编译期确定的用 `char t[] = "hello"`；动态内容用 `memcpy` 拷进自己的缓冲区。裸机平台额外注意：字面量可能落在 flash 里，"只读"由硬件保证，改写同样非法，而且往往不崩、静默失败，更难察觉。顺手立个团队规约：常量字符串表统一写成 `static const char * const tags[] = { ... }`，第二个 const 把指针本身也钉死，连表都不许改。

到这里，指针系列的前七篇收线。回看主线：篇 1 立住"数组名不是指针"，篇 2 给出四要素工具，篇 3 破掉步长陷阱，篇 4 穿透传参本质，篇 5 补上赋值资格审查，篇 6 让行为成为数据，今天划下只读的边界。七个场景反复用的是同一套四问：它是什么类型？指向什么？值是什么？存储在哪？四问问下来，指针没有玄学。指针的故事还没有讲完——出了只读区，下一站是堆，那块 free 之后没人认领的内存。

写在最后

1. `char *s = "hi"` 与 `char t[] = "hi"` 是两种初始化：前者指针指向只读字面量，后者栈上数组持有副本。`sizeof` 立判（8 对 6）。
2. C 的字符串字面量类型是 `char[N]`、没有 const，本体是静态存储期的匿名数组（C11 §6.4.5）；修改它是 UB。C++ 定为 `const char[N]`，"字面量转 `char *`"在 C++ 里也被弃用。
3. 只读的两层防线：`.rodata` 只读页保护（运行期硬件兜底）、相同字面量可能被合并（改一处动全身）。
4. 新代码一律 `const char *s`；接口形参用 `const char *`（`strcpy` 原型同款）；存量代码用 `-Wwrite-strings` 做一次体检。
5. 要可写副本就 `char t[] = "hi"` 或 `memcpy`；裸机平台字面量可能在 flash 里，改写静默失败，比崩溃更难查。

指针系列还在继续。下一篇《free 之后那块内存里有什么》：`free(p)` 之后那块堆内存被清零了吗？里面是 0 还是旧数据？为什么 glibc 急着把它复用出去？指针的深水区，从这块没人认领的内存开始。

合集持续更新，往期与后续篇目都在目录里：C 语言底层实战 · 合集目录

后台回复「交流」，我很乐意分享我的学习计划和一线工作经验，直接来聊。